

Lista de Exercícios – Padrões de Projeto (Criacionais e Estruturais)

1. Builder – Construção de Personagens em um Jogo

Crie um sistema para a criação de personagens em um jogo RPG. O personagem pode ter diferentes classes (Mago, Guerreiro, Arqueiro), armas e habilidades. Use o padrão **Builder** para permitir a criação de personagens passo a passo, garantindo flexibilidade na construção dos atributos.

Desafio: Permitir a criação de personagens a partir de diferentes configurações predefinidas (ex: "Mago Supremo" já vem com cajado e feitiços avançados).

2. Factory Method – Cadastro de Notificações

Implemente um sistema que gerencie notificações via **Factory Method**. O sistema deve permitir a criação de notificações de diferentes tipos (E-mail, SMS e Push Notification). Cada tipo deve ter um método `enviar()`, e a fábrica deve ser capaz de criar objetos da classe correta com base na entrada do usuário.

Desafio: Adicionar um novo tipo de notificação sem modificar a fábrica principal.

3. Facade – Gerenciamento de Pedidos em um Restaurante

Crie um sistema para gerenciar pedidos em um restaurante, onde há várias classes envolvidas (Pedido, Cozinha, Entrega, Pagamento). Utilize **Facade** para fornecer uma interface simples que permita ao cliente fazer um pedido sem se preocupar com a complexidade interna do processo.

Desafio: Implementar um sistema de acompanhamento do pedido que informa em qual etapa ele está (Preparação, Pronto, Saiu para entrega).

4. Composite – Estrutura de Diretórios e Arquivos

Implemente um sistema que represente uma estrutura de arquivos e diretórios utilizando o padrão **Composite**. Um diretório pode conter arquivos e outros diretórios, e cada item (arquivo

ou diretório) deve ser tratado de forma uniforme, permitindo operações como listar conteúdo e calcular tamanho total.

Desafio: Adicionar permissões de leitura e escrita para arquivos e diretórios, aplicando-as de forma hierárquica.

5. Bridge – Dispositivos e Controles Remotos

Implemente um sistema para controlar dispositivos eletrônicos (TV, Rádio e Projetor) usando diferentes tipos de controles remotos (Básico e Avançado). Utilize o padrão **Bridge** para permitir que novos dispositivos ou novos controles possam ser adicionados sem modificar as classes existentes.

Desafio: Criar um controle avançado que adicione funcionalidades como modo silencioso e configuração de timer para desligamento.

6. Decorator – Personalização de Bebidas em uma Cafeteria

Implemente um sistema de personalização de bebidas em uma cafeteria utilizando **Decorator**. O cliente pode escolher uma bebida base (Café, Chá, Chocolate Quente) e adicionar ingredientes extras como leite, chantilly, caramelo e chocolate. Cada extra altera o preço final da bebida.

Desafio: Implementar um sistema de cupons de desconto que reduz o preço final do pedido sem modificar as classes das bebidas.

7. Factory Method – Fábrica de Veículos

Implemente um sistema de fabricação de veículos utilizando o padrão **Factory Method**. A fábrica deve ser capaz de criar diferentes tipos de veículos, como **Carro** e **Caminhão**, podendo ser elétricos ou a combustão. Cada veículo deve ter comportamentos específicos, como **dirigir** para carros e **transportar** para caminhões.

Requisitos:

- Criar uma interface `Veiculo` com métodos comuns.
- Criar classes concretas para os veículos:
 - `CarroEletrico` e `CarroCombustao` devem implementar o método `dirigir()`.

- CaminhaoEletrico e CaminhaoCombustao devem implementar o método transportar().
- Criar uma **fábrica de veículos** (FabricaVeiculo) com o método criarVeiculo(), que retorna uma instância do tipo correto.
- Implementar fábricas concretas para cada tipo de veículo (FabricaCarro e FabricaCaminhao).

Desafio:

Adicionar um novo tipo de veículo, como **Motos**, sem modificar as fábricas existentes.